

TOWARD VERIFIABLE NN4SYS: CERTIFIED ENVIRONMENT TRANSITION MODELING

Xinyi Wei, Gehan Jia

Department of Computer Science
University of Illinois Urbana-Champaign
Champaign, IL 61820, USA
{xwei16, gjia2}@illinois.edu

ABSTRACT

Neural Networks for Systems (NN4Sys) have demonstrated strong empirical performance as controllers in computer systems, yet their vulnerability to adversarial inputs poses fundamental obstacles to safe deployment. Verifying such controllers requires an accurate, trustworthy model of the environment dynamics with which they interact. This work addresses this requirement in the context of adaptive bitrate (ABR) video streaming by constructing a formally bounded neural predictor for the environment transition function f . Using real-world network traces from the Puffer dataset, we train neural networks to estimate next-chunk downloading time from 19 system features and derive rigorous confidence guarantees on predictive error via Clopper–Pearson intervals. Our **best model**, trained on combined multi-day video streaming traces, achieves less than 10% normalized error on at least 98% of inputs with 90% confidence, providing a statistically certified approximation (f^*) of f . This formally bounded transition model addresses a critical gap: while prior work has demonstrated vulnerabilities in ML-based ABR controllers like Pensieve—showing Quality of Experience (QoE) degradation of 30–40% under minor perturbations—our approach establishes the foundation for principled verification of such controllers within a neuro-symbolic framework. This work demonstrates the feasibility of integrating statistically sound environment models into verifiable NN4Sys, enabling formal reasoning about system-level properties such as QoE guarantees.

1 INTRODUCTION

Online video accounts for the majority of internet traffic today. To deliver smooth playback, streaming services dynamically adjust video quality based on current network conditions using adaptive bitrate (ABR) controllers. Conventional ABR schemes rely on hard-coded rules: they monitor recent buffer occupancy and throughput, then apply fixed thresholds to select the bitrate of the next video segment. While simple and interpretable, these heuristics leave substantial QoE improvements unrealized.

Modern controllers replace hand-crafted heuristics with learned policies. Systems such as *Pensieve* Mao et al. (2017) and *Fugu* Yan et al. (2020) train neural networks to maximize QoE-related rewards, enabling players to discover nuanced trade-offs between bitrate, rebuffering risk, and quality variation. In large-scale evaluations, these ML-based controllers deliver higher average quality, fewer stalls, and smoother playback than rule-based baselines—results that have accelerated their adoption across commercial streaming platforms.

However, this performance comes at a cost. Unlike rule-based models, neural networks operate as black boxes with opaque decision boundaries, making their behavior difficult to interpret or verify—especially under adversarial conditions. In computer vision, over a decade of research on *adversarial examples* has demonstrated that small, carefully crafted perturbations can cause models to produce confident but incorrect predictions Szegedy et al. (2014). Similar vulnerabilities affect NN4Sys.

Prior work has shown that ABR controllers are similarly vulnerable. Small, targeted perturbations to easily accessible input features—such as the most recent download time—can substantially de-

grade QoE without requiring access to controller internals. Using Projected Gradient Descent (PGD) within specification-constrained bounds generated from *SpecTRA* Chaudhary et al. (2024), we previously demonstrated QoE reductions of 30–40% through practical, low-cost network-layer attacks. While these results highlight clear vulnerabilities, empirical demonstrations alone provide limited insight into *why* failures occur or *how* to design principled defenses.

This work. We advance toward verifiable ML-based system controllers by formulating the ABR pipeline as a neuro-symbolic system and constructing a formally bounded environment model. Specifically, we approximate the environment transition function f —which maps a 19-dimensional feature vector of network conditions and chunk metadata to next-chunk download time—using a neural network trained on real-world traces from the Puffer dataset. To establish formal reliability guarantees, we derive Clopper–Pearson confidence intervals on the predictor’s error distribution, yielding statistically certified bounds on f ’s approximation quality. Our best model achieves less than 10% normalized error on at least 98% of inputs with 90% confidence.

These guarantees provide a critical building block for controller verification: a bounded and trustworthy environment model. Building on neuro-symbolic training approaches Yang & Chaudhuri (2022), this certified environment model enables training ABR controllers with symbolic constraints, safety invariants, and verifiable control properties. Our contributions are:

- A neural predictor f^* for the environment transition function f in ABR streaming, trained on real-world network traces with 19 input features;
- Statistically certified error bounds via Clopper–Pearson intervals, providing formal guarantees on prediction accuracy;
- A methodology for constructing formally bounded environment models in NN4Sys, establishing the foundation for future controller verification.

2 BACKGROUND AND MOTIVATION

This section provides background on ML-based ABR controllers and motivates the need for verifiable system designs. We first describe how video streaming operates and the role of ABR controllers (§2.1). We then discuss Pensieve, a state-of-the-art neural ABR controller (§2.2), before examining its vulnerabilities to adversarial perturbations (§2.3). Finally, we outline the requirements for verifiable controller design (§2.4).

2.1 VIDEO STREAMING AND ABR CONTROLLERS

Online video has become the dominant load on the public Internet. To keep playback smooth over volatile links, nearly all modern streaming players employ an *ABR* controller. Video content is delivered in discrete chunks (typically 2–4 seconds each), and after downloading each chunk, the client uses recent network metrics—such as throughput and buffer occupancy—to determine the quality level of the next chunk.

Conventional ABR schemes—such as *BOLA* Spiteri et al. (2020) and *MPC* Yin et al. (2015)—encode expert-designed rules. For example: “maintain at least B seconds of buffered video, then select the highest bitrate whose predicted download time is less than $B - C$.”¹ While these heuristics perform reasonably well under typical conditions, they struggle to capture the complex, nonlinear relationships between bandwidth variability, buffer dynamics, and chunk size.

The primary objective of ABR controllers is to maximize QoE, which balances three factors:

- **Video quality:** Higher bitrates improve visual quality but increase download time and rebuffering risk;
- **Rebuffering:** Playback stalls severely degrade user experience;
- **Quality switches:** Frequent bitrate changes are perceptually jarring.

¹The constants B and C are typically hand-tuned for each deployment.

A typical QoE metric combines these factors as:

$$\text{QoE} = \sum \text{bitrate} - \lambda_1 \cdot \sum \text{rebuffer time} - \lambda_2 \cdot \sum |\Delta \text{bitrate}|,$$

where λ_1 and λ_2 are penalty weights.

2.2 MACHINE LEARNING FOR ABR: PENSIEVE

Pensieve Mao et al. (2017) addresses the limitations of rule-based ABR by replacing hand-crafted heuristics with a neural policy trained via reinforcement learning. The model selects a bitrate for each video chunk based on recent network and playback state. At each decision point, the player constructs a state representation $\mathbf{S}$ consisting of six types of features, each with eight historical values:

1. past throughput samples,
2. past download times,
3. past buffer occupancy levels,
4. past selected bitrates,
5. available chunk sizes at all quality levels,
6. number of chunks remaining until video end.

This produces a 6×8 matrix, which is flattened into a 48-dimensional state vector and fed into a two-layer fully connected neural network. The network outputs a softmax distribution over six discrete bitrate actions $a \in \{300, 750, 1200, 1850, 2850, 4300\}$ kbps. The model is trained to maximize a QoE-based reward function:

$$R = R_{\text{bitrate}} - 4.3 \cdot R_{\text{rebuffer}} - R_{\text{switch}},$$

where R_{bitrate} is the cumulative video quality utility, R_{rebuffer} is the total rebuffering time, and R_{switch} represents bitrate variation penalties. In empirical evaluations, Pensieve achieves QoE gains of 10–70% over traditional controllers Mao et al. (2017), motivating widespread interest in ML-based ABR approaches.

2.3 ADVERSARIAL VULNERABILITIES IN NN4SYS

Despite their strong empirical performance, neural network-based system controllers can be highly sensitive to small, structured perturbations. Recent work has demonstrated vulnerabilities in various NN4Sys applications, and ABR controllers are no exception.

Our prior work performed adversarial attacks on Pensieve, showing that neural ABR controllers can be compromised even under mild, in-distribution feature manipulations. We implemented a PGD attack that perturbs only a single input feature—`last_download_time`—by at most 10%. Despite this constraint, the attack successfully forces Pensieve to select suboptimal bitrates, decreasing average bitrate by 35–65% and reducing QoE by 30–40% across attacked sessions.

Critically, these perturbations remain within empirically valid ranges and are achievable through realistic network-layer manipulations (e.g., delaying packet delivery slightly). They evade basic sanity checks and thus pose a realistic security threat for NN4Sys deployments. These findings underscore the brittleness of black-box neural policies and motivate the development of verifiable training and deployment pipelines.

2.4 TOWARD VERIFIABLE ABR CONTROLLERS

The vulnerabilities demonstrated above highlight a fundamental challenge: ML-based controllers operate in a closed-loop setting where a neural policy observes the current state, selects an action, and an environment transition function produces the next state. This structure can be viewed as a *neuro-symbolic system*, where the controller is a learned neural component and the environment dynamics form the symbolic component.

In ABR streaming, the neural controller (e.g., Pensieve) outputs bitrate decisions, while the environment transition function f determines how these decisions interact with network conditions to shape future states and QoE. Specifically, f takes as input the current system state (buffer occupancy, recent throughput, etc.) and the selected bitrate, then produces the next state—including the critical but unpredictable download time of the next chunk.

Verifying such controllers requires not only analyzing the neural policy but also obtaining a reliable and bounded approximation of f . Without a trustworthy environment model, any formal guarantees about controller behavior remain incomplete. This is the gap our work addresses: we construct a formally bounded neural predictor f^* that approximates the environment transition function with statistically certified error bounds, providing the foundation needed for downstream controller verification.

3 METHODOLOGY

This section describes our approach to constructing a formally bounded environment model for ABR streaming systems. We first formulate the ABR pipeline as a neuro-symbolic system (§3.1), then detail the neural network architecture used to approximate the environment transition function (§3.2), and finally explain how we derive statistical certification guarantees via Clopper–Pearson bounds (§3.3).

3.1 NEURO-SYMBOLIC SYSTEM FORMULATION

We model adaptive bitrate (ABR) streaming as a *neuro-symbolic* closed-loop system composed of two interacting components: (1) a neural controller that selects bitrates based on the current state, and (2) an environment transition function that determines how each bitrate decision influences future network and playback behavior. This decomposition captures both the expressive, data-driven decision logic of the controller and the structured evolution of the underlying system state, enabling the application of verification techniques to ML-based control pipelines.

System dynamics. Let s_t denote the system state at time t , consisting of recent throughput measurements, download times, buffer occupancy, past bitrates, available chunk sizes, and remaining chunks. The neural controller (Pensieve in our case) selects a bitrate $a_t = \text{NN}(s_t)$ based on this state. The environment transition function f then maps (s_t, a_t) to the next state s_{t+1} :

$$s_{t+1} = f(s_t, a_t).$$

The function f embodies complex network characteristics, including bandwidth fluctuations, queuing delay, and chunk sizes, whose exact form is unknown and varies with network conditions. The key challenge is that while we can observe state transitions in real traces, we cannot directly access or verify f itself. This makes closed-loop verification intractable without a reliable approximation.

The approximation challenge. To enable verification, we need a trustworthy approximation f^* of the true environment transition function f . However, the download time component of f —which determines buffer dynamics and rebuffering risk—depends on unpredictable network conditions and cannot be determined until a chunk is actually downloaded. We address this by training a neural network f^* to predict next-chunk download time from observable system features, then deriving formal statistical guarantees on its prediction accuracy.

Closed-loop control with safety assertions. Algorithm 1 formalizes the closed-loop interaction between the neural controller and the certified environment model. The system maintains a running estimate of average QoE and tracks consecutive violations of a minimum acceptable threshold α . The assertion at line 11 enforces that either QoE remains above α , or the number of consecutive violations stays below a tolerance limit β . This formulation allows us to reason about temporal safety properties while accommodating transient QoE fluctuations that may occur under challenging network conditions.

Here, NN is the Pensieve controller, s_0 is the initial streaming session state, α is the minimum acceptable average QoE threshold, and β bounds the maximum allowed consecutive QoE violations.

Algorithm 1 Neuro-symbolic formulation of ABR streaming

```

1:  $a \leftarrow \text{NN}(s_0)$  {Initial action from Pensieve given initial state  $s_0$ }
2:  $s \leftarrow f^*(s_0, a)$  {Environment transition via certified predictor}
3:  $\text{failure\_counter} \leftarrow 0$ 
4: while True do
5:    $a \leftarrow \text{NN}(s)$  {Pensieve outputs next bitrate decision}
6:    $s \leftarrow f^*(s, a)$  {Update state using certified environment model}
7:   if  $\text{QoE}_{\text{avg}} < \alpha$  then
8:      $\text{failure\_counter} \leftarrow \text{failure\_counter} + 1$ 
9:   else
10:     $\text{failure\_counter} \leftarrow 0$ 
11:   end if
12:   assert  $(\text{QoE}_{\text{avg}} \geq \alpha) \vee (\text{failure\_counter} < \beta)$ 
13: end while

```

Constructing a certified approximation f^* with quantifiable error guarantees is the central goal of this work.

3.2 ENVIRONMENT MODEL ARCHITECTURE

Dataset and features. To train the environment predictor f^* , we use real-world ABR traces collected from the Puffer dataset Yan et al. (2020). Each training sample represents a state-action pair and contains 19 features:

- 8 past bandwidth measurements (in Mbps),
- 8 past download times (in seconds),
- 8 past chunk sizes (in MB),
- The upcoming chunk size (in MB).

These inputs form a 19-dimensional feature vector used to predict the (normalized) next-chunk download time. This empirical data provides a realistic basis for modeling environment transitions in ABR streaming under diverse network conditions.

Neural network architecture. The environment predictor f^* is implemented as a fully connected neural network. We experimented with both 2-hidden layer and 3-hidden layer architectures. The output is a single linear regression value corresponding to the predicted next-chunk download time. The network is trained using the Adam optimizer with L1 loss, which improves robustness to heavy-tailed download-time fluctuations common in real network traces.

Normalization. All features and labels are normalized to the range $[0, 1]$ by dividing by their respective maximum values observed in the training set. Training, evaluation, and certification are performed entirely in this normalized space. This approach ensures numerical stability and allows error guarantees to be expressed in terms of relative prediction error, consistent with prior work on ABR trace modeling.

Training procedure. We trained two types of models to evaluate generalization:

1. **Individual models:** 10 models, each trained on 1,000 samples from a single day and tested on different 1,000 samples from the same day.
2. **Combined model:** One model trained on 10,000 samples drawn from 10 random days (1,000 per day) and tested on 9,000 samples from another 9 different random days.

The combined model evaluates generalization across diverse network conditions.

3.3 STATISTICAL CERTIFICATION VIA CLOPPER–PEARSON BOUNDS

To provide formal reliability guarantees, we employ Clopper–Pearson confidence intervals Clopper & Pearson (1934), which offer exact binomial confidence bounds without parametric assumptions.

Success criterion. A prediction is considered a *success* if it satisfies:

$$|f(x) - f^*(x)| \leq \epsilon,$$

where $f(x)$ is the ground-truth next-chunk download time, $f^*(x)$ is the predicted value, and $\epsilon = 0.10$ represents a 10% tolerance in the normalized domain. This threshold balances precision with practical robustness requirements for ABR systems.

Confidence interval computation. Given a test set of n samples, suppose k samples satisfy the success criterion. We model each prediction as an independent Bernoulli trial with unknown success probability p . The Clopper–Pearson method computes an exact confidence interval for p using the cumulative distribution function of the binomial distribution:

$$\text{CI}_{1-\alpha}(p) = \left[\text{Beta}^{-1} \left(\frac{\alpha}{2}; k, n - k + 1 \right), \text{Beta}^{-1} \left(1 - \frac{\alpha}{2}; k + 1, n - k \right) \right],$$

where α is the significance level (we use $\alpha = 0.10$ for 90% confidence) and Beta^{-1} denotes the inverse beta cumulative distribution function. The lower bound of this interval provides a conservative estimate of the predictor’s success rate.

Interpretation. For our best model (the *combined model*), the 90% Clopper–Pearson confidence interval on test set accuracy is $[0.9846, 0.9887]$. We use the lower bound as our certified guarantee:

$$\Pr(|f(x) - f^*(x)| \leq 0.10) \geq 0.9846 \quad \text{with 90\% confidence.}$$

This conservative bound ensures that, with 90% confidence, at least 98.46% of future predictions will have less than 10% normalized error under the i.i.d. assumption.

Key assumption. The validity of Clopper–Pearson bounds requires that test samples are independent and identically distributed (i.i.d.). We satisfy this assumption by: (1) randomly shuffling trace data before train-test splitting, and (2) ensuring temporal independence by drawing samples from different streaming sessions and days. While real network traces exhibit temporal correlations within sessions, our sampling strategy mitigates this concern by treating each chunk download as an independent observation.

Providing a certified environment model. The statistical guarantee derived from Clopper–Pearson bounds serves as the formal foundation for using f^* in verification tasks. Unlike network simulators deployed without guarantees, our certified predictor comes with quantifiable reliability: we know, with high confidence, what fraction of predictions will be accurate. This transforms f^* from an heuristic model into a trustworthy symbolic component suitable for reasoning about closed-loop system behavior and enabling principled verification of neural ABR controllers.

4 EVALUATION

We evaluate our neural environment predictors on real-world network traces from the *Puffer* dataset. Our evaluation focuses on two complementary objectives: (1) measuring raw prediction accuracy, and (2) deriving formal statistical guarantees suitable for verification. We train models on individual daily traces and on a combined multi-day dataset to assess both in-distribution performance and generalization across diverse network conditions.

4.1 EVALUATION METRICS

We evaluate model reliability through statistical certification using exact binomial confidence intervals. Because our goal is a *verifiable* environment model suitable for formal reasoning, we focus on deriving provable guarantees on prediction accuracy rather than traditional point-wise error metrics.

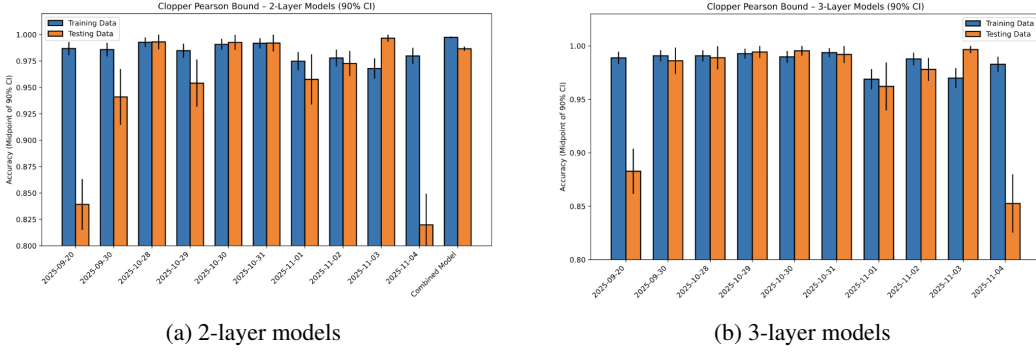


Figure 1: Clopper–Pearson certified accuracy for 2-layer and 3-layer models trained on individual daily traces and a combined multi-day dataset. Error bars show confidence interval half-width.

A prediction is counted as a *success* if:

$$|f(x) - f^*(x)| \leq 0.10,$$

i.e., the prediction error is within 10% in normalized space. Given the observed success rate on a test set, we compute the 90% Clopper–Pearson confidence interval $[L, U]$ on the true success probability. We report the interval midpoint $(L + U)/2$ with error bars of width $(U - L)/2$ to visualize both the point estimate and the statistical uncertainty. The lower bound L provides our certified guarantee.

4.2 EXPERIMENTAL SETUP

We trained two types of models:

- **Individual models:** 10 models per architecture, each trained on 1,000 samples from a single day and tested on 1,000 held-out samples from the same day. These models evaluate in-distribution performance.
- **Combined model:** One model per architecture trained on 10,000 samples drawn from 10 different days and tested on 9,000 samples from 9 different days. This model evaluates cross-day generalization.

All models use the architectures described in §3.2, trained with the Adam optimizer and L1 loss. We evaluate both 2-hidden layer and 3-hidden layer networks.

4.3 RESULTS

Figure 1 shows the 90% Clopper–Pearson certified accuracies for both architectures. Error bars indicate confidence interval half-width.

Overall performance. Across both architectures, certified training accuracies remain high (typically 0.98–0.995), indicating that all models reliably fit their training distributions. Test performance varies more substantially across individual traces, reflecting the inherent non-stationarity of real network environments. Most models achieve certified test accuracies above 0.95, demonstrating that the predictors generalize well within the 10% error tolerance.

Architecture comparison. The 3-layer models generally achieve tighter confidence intervals and slightly higher certified accuracies on traces from late October to early November. For instance, on traces from October 30–31, 2024, the 3-layer models reach certified accuracies of approximately 0.99, compared to 0.985–0.99 for 2-layer models. This suggests that additional capacity helps capture complex temporal patterns in network behavior.

However, deeper models also show larger degradation on challenging traces. On September 20 and November 4, 2024, the 3-layer models achieve certified accuracies of roughly 0.88 and 0.85, respectively—slightly lower than their 2-layer counterparts. This indicates mild overfitting to training distributions, a known limitation of deeper networks on heterogeneous data.

Temporal stability. Both architectures perform best on traces from late October through early November (e.g., October 28–November 1, 2024), achieving certified accuracies in the 0.96–0.99 range with narrow confidence intervals. These traces likely represent more stationary network conditions. Performance degrades on earlier traces (mid-September), suggesting distribution shift or higher network variability during those periods.

Combined model performance. The combined model—trained on data from 10 different days—achieves strong and stable certified accuracy. The 2-layer combined model achieves a lower bound of 0.9846 with 90% confidence (§3.3), meaning at least 98.46% of predictions have less than 10% error. Importantly, the combined model exhibits one of the tightest Clopper–Pearson intervals across all experiments, making it the most reliable candidate for use as a certified environment predictor in verification tasks.

Key findings. Our evaluation demonstrates that:

- Both 2-layer and 3-layer models achieve $> 95\%$ certified accuracy under 10% error tolerance on most traces;
- 3-layer models provide higher peak accuracy but are more sensitive to distribution shift;
- 2-layer models offer greater stability on challenging traces;
- Combined models trained on diverse daily traces provide the best balance of accuracy and confidence interval tightness.

These findings validate that a diverse training set and modest network capacity yield reliable, formally bounded environment predictors suitable for verifiable ABR control.

5 RELATED WORK

ML-based ABR controllers. Pensieve Mao et al. (2017) introduced reinforcement learning for ABR control, demonstrating significant QoE improvements over rule-based methods. Subsequent work, including Fugu Yan et al. (2020) and other RL-based controllers, extended this approach across diverse streaming scenarios.

Adversarial vulnerabilities in NN4Sys. Recent studies have exposed the adversarial fragility of neural system controllers, showing that small input perturbations can severely degrade performance. These findings motivate the need for verifiable controller designs with formal robustness guarantees.

Formal verification and statistical certification. Formal methods for neural network verification have gained traction in safety-critical domains Singh et al. (2019). Statistical certification techniques, including Clopper–Pearson intervals Clopper & Pearson (1934) and PAC-style generalization bounds, provide rigorous guarantees without exhaustive symbolic analysis. Our work applies these techniques to environment modeling in NN4Sys, addressing a key prerequisite for verification.

6 CONCLUSION

This paper presents a statistically certified neural predictor for environment transitions in adaptive bitrate streaming systems. We formulate the ABR pipeline as a neuro-symbolic system and construct a formally bounded approximation of the environment transition function using real-world network traces from the Puffer dataset. By applying Clopper–Pearson confidence intervals, we derive rigorous statistical guarantees on prediction accuracy: our best model achieves less than 10% normalized error on at least 98.46% of inputs with 90% confidence.

These results demonstrate that formally bounded environment models are both feasible and accurate enough for verification tasks. Our certified predictor addresses a critical prerequisite for verifying ML-based ABR controllers in closed-loop settings, providing the trustworthy symbolic component needed for principled reasoning about system behavior. This work establishes a methodology for integrating statistical certification into NN4Sys, paving the way for future development of verifiable neural controllers with formal guarantees on QoE and robustness properties.

REFERENCES

- Isha Chaudhary, Shuyi Lin, Cheng Tan, and Gagandeep Singh. Specification generation for neural networks in systems, 2024. URL <https://arxiv.org/abs/2412.03028>.
- Charles J Clopper and Egon S Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.
- Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication, SIGCOMM '17*, pp. 197–210, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450346535. doi: 10.1145/3098822.3098843. URL <https://doi.org/10.1145/3098822.3098843>.
- Gagandeep Singh, Timon Gehr, Markus Püschel, and Martin Vechev. An abstract domain for certifying neural networks. *Proc. ACM Program. Lang.*, 3(POPL), January 2019. doi: 10.1145/3290354. URL <https://doi.org/10.1145/3290354>.
- Kevin Spiteri, Rahul Uргаonkar, and Ramesh K. Sitaraman. Bola: Near-optimal bitrate adaptation for online videos. *IEEE/ACM Transactions on Networking*, 28(4):1698–1711, 2020. doi: 10.1109/TNET.2020.2996964.
- Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks, 2014. URL <https://arxiv.org/abs/1312.6199>.
- Francis Y. Yan, Hudson Ayers, Chenzhi Zhu, Sadjad Fouladi, James Hong, Keyi Zhang, Philip Levis, and Keith Winstein. Learning in situ: a randomized experiment in video streaming. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pp. 495–511, Santa Clara, CA, February 2020. USENIX Association. ISBN 978-1-939133-13-7. URL <https://www.usenix.org/conference/nsdi20/presentation/yan>.
- Chenxi Yang and Swarat Chaudhuri. Safe neurosymbolic learning with differentiable symbolic execution, 2022. URL <https://arxiv.org/abs/2203.07671>.
- Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. A control-theoretic approach for dynamic adaptive video streaming over http. *SIGCOMM Comput. Commun. Rev.*, 45(4):325–338, August 2015. ISSN 0146-4833. doi: 10.1145/2829988.2787486. URL <https://doi.org/10.1145/2829988.2787486>.